

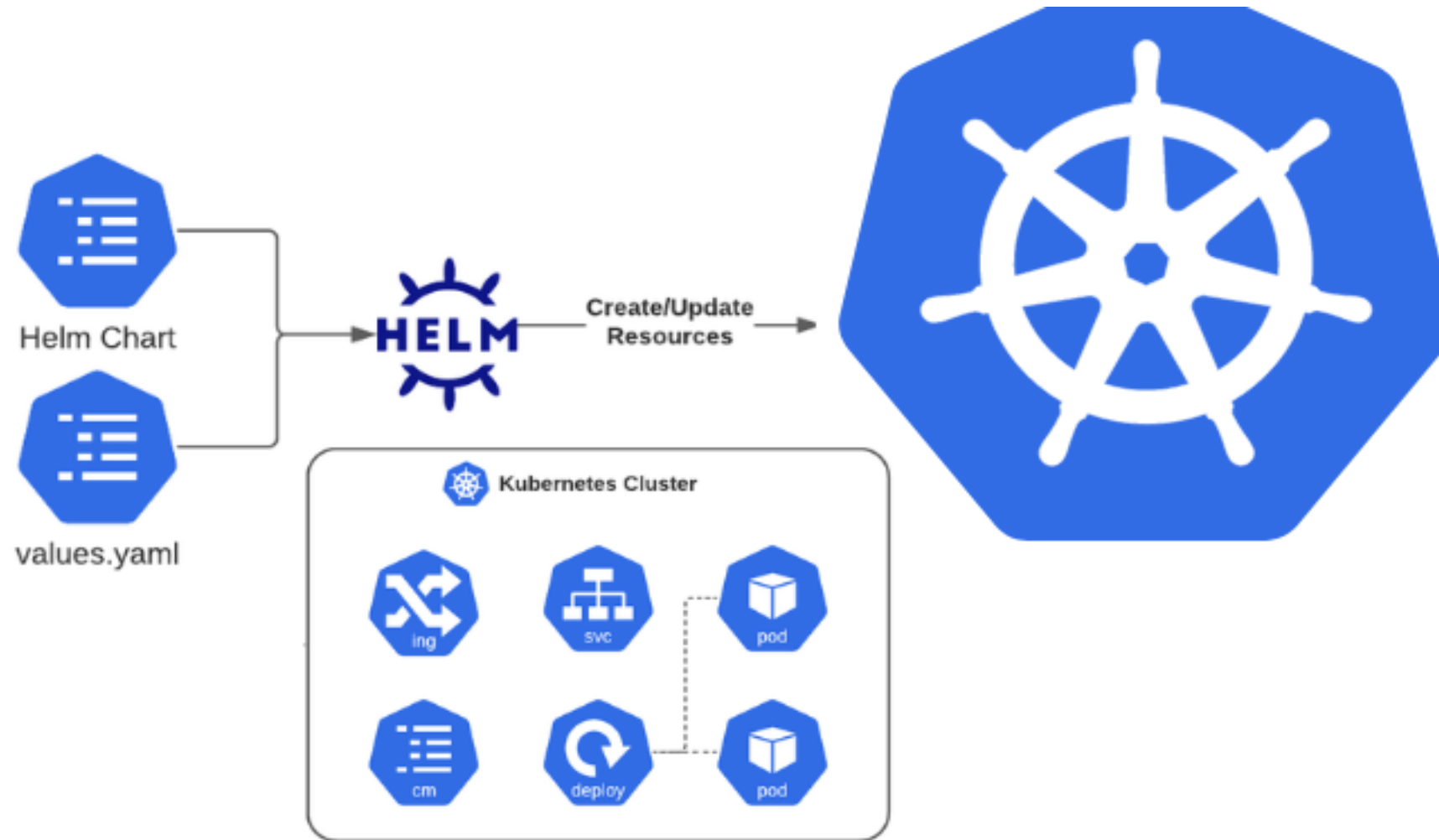
CI/CD-Pipelines am Beispiel von GitLab Ausblick helm-Grundlagen

Container mit Schwerpunkt
Kubernetes



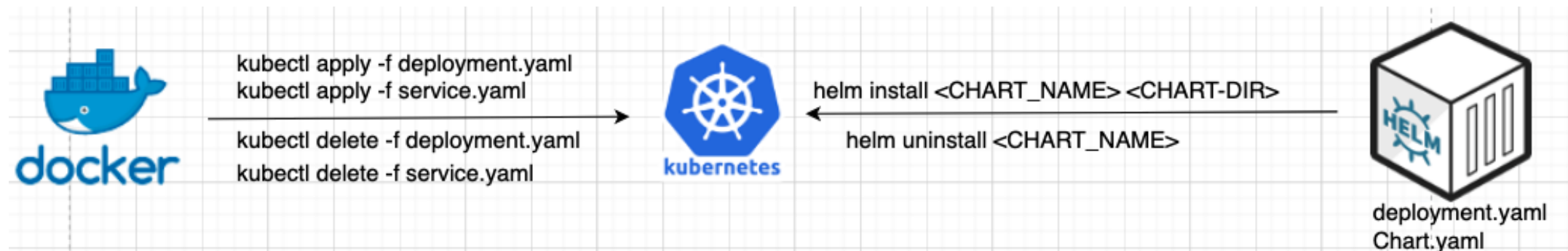
- Helm Überblick
- Helm für die Installation verwenden
- Helm-Charts erstellen

Paketmanager für Kubernetes - Helm



Was ist Helm?

- Helm erleichtert das Rollout einer Anwendung.
- Alle Kubernetes-Objekte werden in einem **Helm-Chart** zusammengefasst.
- Die Installation eines Helm-Charts erfolgt durch einen Befehl.
- Die Anpassung des Rollouts an die Umgebung erfolgt über Parameter der Datei **values.yaml**.
- Die Erstellung von Helm-Charts setzt Kenntnisse von Kubernetes voraus.



- **Helm Chart (Paket)**
ist ein Paket, das alles enthält, was notwendig ist, um eine Anwendung in Kubernetes zu installieren. Es umfasst Templates, Konfigurationsdateien, Abhängigkeiten und Metadaten.
- **Helm Release**
ist eine instanziierte Version eines Charts, die in einem Kubernetes-Cluster installiert ist.
- **Helm Repository**
ist eine Sammlung von Charts, die an einem zentralen Ort gespeichert sind. Es gibt eine Vielzahl von Helm-Charts für die Installation diverser Software (Datenbanken, Application-Server, Middleware,...). Suchmaschine für Helm-Charts ist <https://artifacthub.io>
- **Helm Client (CLI)**
ist das Kommandozeilenwerkzeug, mit dem Sie Helm verwenden können.

Agenda

- Helm Überblick
- Helm für die Installation verwenden
- Helm-Charts erstellen

- Helm-Repositories stellen Helm-Charts zur Verfügung.
- Das CLI für Helm ist *helm*. 😊
- Auch für *helm* gibt es eine Autovervollständigung: *source <(helm completion bash)*
- Helm-Repositories müssen dem Client *helm* bekannt gemacht werden.
helm repo add bitnami https://charts.bitnami.com/bitnami
- Der Index eines Helm-Repositories wird wie folgt geladen.
helm repo update
- Mit *repo add* fügen Sie ein Alias für ein Repository hinzu.
- Mit *repo delete* Löschen Sie ein Alias für ein Repository.
- Auflisten der bekannt gemachten Helm-Chart-Repositories.

```
# helm repo list
NAME      URL
bitnami   https://charts.bitnami.com/bitnami
```

- *# helm repo update*: aktualisiert den lokalen Cache für die Indexe der ihm bekannten Repositories.
- Nach Helm-Charts suchen.

helm search repo wildfly

NAME	CHART VERSION	APP VERSION	DESCRIPTION
bitnami/wildfly	21.0.5	33.0.2	Wildfly is a lightweight, open source...

- Mehrere Versionen anzeigen lassen.

helm search repo wildfly --versions

NAME	CHART VERSION	APP VERSION	DESCRIPTION
bitnami/wildfly	21.0.6	33.0.2	Wildfly is a lightweight, open source applicati...
bitnami/wildfly	21.0.5	33.0.2	Wildfly is a lightweight, open source applicati...
bitnami/wildfly	21.0.4	33.0.1	Wildfly is a lightweight, open source applicati...
bitnami/wildfly	21.0.3	33.0.0	Wildfly is a lightweight, open source applicati...

- *CHART VERSION*: Version des Helm-Charts.
- *APP VERSION*: Version der Anwendung.

Befehle für das Ausrollen von Helm-Charts

- **Installation eines Charts:**
helm install <release-name> <chart>
- **Aktualisierung eines Releases:**
helm upgrade <release-name> <chart>
- **Rollback eines Releases:**
helm rollback <release-name> <revision>
- **Listing der Releases:**
helm list
- **Deinstallation eines Releases:**
helm uninstall <release-name>

```
# helm repo list
NAME      URL
bitnami   https://charts.bitnami.com/bitnami
# helm repo update
...Successfully got an update from the
"bitnami" chart repository
Update Complete. ✖Happy Helming!✖
# helm install bitnami/mysql \
    --generate-name
NAME: mysql-1729104595
LAST DEPLOYED: Wed Oct 16 20:49:58 2024
NAMESPACE: user02-namespace
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: mysql
CHART VERSION: 11.1.19
APP VERSION: 8.4.3
...
```

Informationen zu einem Release

```
# kubectl get all -l app.kubernetes.io/instance=mysql-1729104595
```

NAME	READY	STATUS	RESTARTS	AGE
pod/mysql-1729104595-0	1/1	Running	0	5m14s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/mysql-1729104595	ClusterIP	10.152.183.143	<none>	3306/TCP	5m14s
service/mysql-1729104595-headless	ClusterIP	None	<none>	3306/TCP	5m14s

NAME	READY	AGE
statefulset.apps/mysql-1729104595	1/1	5m14s

- **Auflisten der aktiven Objekte**

- # kubectl get all -l app.kubernetes.io/instance=<Release-Name>*

- **Alle Manifeste zu einem Release ermitteln.**

- # helm get manifest <Release-Name>*

- **Secrets werden dabei nicht angezeigt.**

- Im Helm-Charts sind Parameter in der Datei *values.yaml* hinterlegt.
- Auflisten aller Parameter.
helm get values <Release> --all
- Einzelne Parameter für Release festlegen mit der Option *--set <Key>=<Value>*.
- Option ist für *helm install* oder *helm upgrade* einsetzbar.
- Anpassung vieler Parameter mit einer eigenen *values.yaml*-Datei.
*# helm install -f values.yaml bitnami/mysql *
--generate-name

```
# helm get values mysql-1729104595 --all
COMPUTED VALUES:
architecture: standalone
auth:
  authenticationPolicy: ""
...
# helm upgrade mysql-1729104595 \
  bitnami/mysql \
  --set auth.rootPassword=train4ordix
# helm install -f values.yaml \
  bitnami/mysql \
  --generate-name
```

Agenda

- Helm Überblick
- Helm für die Installation verwenden
- Helm-Charts erstellen

- *Helm Chart* ist ein Paket für eine Kubernetes-Anwendungen.
- Es beinhaltet alle erforderlichen Manifest-Dateien.
- Notwendige Verzeichnis-Struktur:
 - Root-Verzeichnis benannt mit dem Chart-Namen
 - Im Root-Verzeichnis:
 - Chart.yaml: Deklaration des Charts.
 - values.yaml: Parameter für das Chart.
 - Im Verzeichnis Root/templates:
 - Parametrisierte Manifest-Dateien.

- Anlegen eines neuen Helm Charts:
helm create YOUR-CHART-NAME

YOUR-CHART-NAME/

- | | |
|---------------|---|
| | |
| - .helmignore | |
| | |
| - Chart.yaml | - Chart.yaml: |
| | Metadaten zum Helm Chart. |
| - values.yaml | - values.yaml: |
| | Überschreibbare Werte für das Template. |
| - charts/ | - charts: |
| | Verzeichnis mit weiteren anhängigen Charts. |
| - templates/ | - templates: |
| | Verzeichnis mit K8s Manifests |
| | (.yaml-Dateien und .tpl-Datei). |

- Enthält die Metadaten des Charts.
- Auswahl der „wichtigsten“ Felder

Feld	Beschreibung
apiVersion	Mit welcher Helm Version ist der Chart kompatibel (required) • v2: für Charts die Helm 3 erfordern • v1: für Charts die auch mit älteren Helm-Versionen kompatibel sind
Name	Name des Charts (required)
Version	Versionsnummer des Charts (required) • Muss SemVer2 kompatibel sein
Description	Kurze Beschreibung des Charts
dependencies	Liste mit den Abhängigkeiten (Charts) des Charts
appVersion	Version der Applikation die der Chart enthält

Helm Templates sind das Herzstück von Helm Charts und ermöglichen die dynamische Erstellung von Kubernetes-Ressourcen durch die Verwendung der Go-Template-Engine.

- **Templates + Values = Kubernetes-Manifest**
- Alle Dateien im Verzeichnis *templates* werden als **YAML-Templates** behandelt.
- Für die Injektion von Parametern wird die **Go Template Language** eingesetzt.
- Die Parametrisierung macht es möglich, dieselben Templates für unterschiedliche Umgebungen zu verwenden.

Helm-Chart - Hello World (1)

- Chart.yaml

```
apiVersion: v1  
name: helloworld  
version: 0.1.0  
description: print Hello <World>!
```

- values.yaml

```
name: "World"
```

Praxis

- Erstellung eines Helm-Charts

helm create

- Erstellung eines generischen Helm-Charts.
 - Dateien `values.yaml` und `Chart.yaml` werden erstellt.
 - Alle Template-Dateien werden automatisch erstellt.
 - Enthält **generische Template-Dateien**.
 - Anpassungen erfolgen (möglichst) nur in der Datei `values.yaml`.
-
- Sehr guter Ausgangspunkt für generische Helm-Charts.
 - Sehr umfangreiche Parameter-Datei `values.yaml`.

Helm Chart mit *helm create* erstellen

- `helm create mychart`

```
# Chart.yaml  
  
apiVersion: v2  
name: mychart  
description: A Helm chart for Kubernetes  
type: application  
# This is the chart version  
version: 0.1.0  
# This is the version number of the application being deployed.  
appVersion: "1.16.0"
```

```
| -Chart.yaml  
| -charts  
| -.helmignore  
| -templates  
| | -deployment.yaml  
| | -NOTES.txt  
| | -ingress.yaml  
| | -tests  
| | | -test-connection.yaml  
| | -service.yaml  
| | -hpa.yaml  
| | -serviceaccount.yaml  
| | -_helpers.tpl  
| -values.yaml
```

- Applikationsspezifische Daten müssen in **values.yaml** eingepflegt werden.
- Helm Install (--dry-run): `helm install mychart --debug --dry-run mychart`
- Syntaktische Überprüfung: `helm lint mychart`

Anpassung - Chart.yaml

- *description*: Kurzbeschreibung für die Anwendung.
- *version*: Die Version des Helm-Charts anpassen.
- *appVersion*: Version der Anwendung eintragen.

```
apiVersion: v2
name: busybox
description: Example for a helm-chart.
type: application
version: 0.1.0
appVersion: "1.36.1"
```

- *repository*: Registry und Repository des Docker-Images
- *version*: Die Version des Helm-Charts anpassen.
- *appVersion*: Version der Anwendung eintragen - alternativ wird die Version aus Chart.yaml verwendet.

```
image:
  repository: busybox
  pullPolicy: IfNotPresent
  # default is the chart appVersion.
  tag: "1.36.1"
...
service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  className: ""
  annotations: {}
    # kubernetes.io/ingress.class: nginx
    # kubernetes.io/tls-acme: "true"
  hosts:
    - host: chart-example.local
      paths:
        - path: /
          pathType: ImplementationSpecific
```

- Öffentliche Helm-Charts kommen weltweit zum Einsatz.
- Helm-Charts sind vorbereitet für den Einsatz in unterschiedlichen Umgebungen.
- Parameter in den Template-Dateien werden durch Values bei der Installation definiert.
- Template-Dateien lassen sich mit der Go-Template-Language parametrisieren.

Weitere Informationen:

<https://pkg.go.dev/text/template>

- Das Ergebnis der Templates kann direkt ausgegeben werden:
helm template my-release ./my-helm-chart --output-dir ./output --debug
- *--output-dir*: Zielverzeichnis der Manifeste.
- *--debug*: Erweitertes Logging.

Template mit Value-Parameters

- In Templates ist die Logik für die Parameter-Injektion durch `{{...}}` gekennzeichnet.
- `.Values.<Name>`: injiziert den Wert der Variablen `<Name>` aus der Datei `values.yaml`.

```
apiVersion: v1
kind: Pod
metadata:
  name: helloworld
spec:
  containers:
  - name: helloworld
    image: busybox
    command: ['sh', '-c' \
, 'echo Hello {{ .Values.name }}!']
    restartPolicy: Never
```

Templates mit Release-Parameter

Helm erstellt bei der Installation Parameter.
Beispiele:

- Release.Name
 - Release.Namespace
 - Release.IsUpgrade
 - Chart.Name
 - Chart.Version
-
- Namens-Konflikte lassen sich dadurch vermeiden.
 - Referenzen zwischen Labels und Selektoren lassen sich herstellen.

```
apiVersion: v1
kind: Pod
metadata:
  name: {{.Release.Name}}
  labels:
    app: {{.Release.Name}}
spec:
  containers:
    - name: example-container
      image: nginx:latest
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: {{.Release.Name}}
spec:
  type: ClusterIP
  selector:
    app: {{.Release.Name}}
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Das Minus-Zeichen entfernt Leerzeichen vor oder Nach einem Wert.

- `{{ ... }}`: Standard, Leerzeichen bleiben erhalten.
- `{{- ... }}`: Entfernt Leerzeichen vor der Anweisung.
- `{{ ... -}}`: Entfernt Leerzeichen nach der Anweisung.
- `{{- ... -}}`: Entfernt Leerzeichen vor und nach der Anweisung.
- Das Minuszeichen in logischen Ausdrücken eliminiert Leerzeilen.

```
{{- if .Values.ingress.enabled -}}
{{- $fullName := include
"test.fullname" . -}}
{{- $svcPort := .Values.service.port
-}}
{{- if and .Values.ingress.className
(not (semverCompare ">=1.18-0"
.Capabilities.KubeVersion.GitVersion
)) }}
    {{- if not (hasKey
.Values.ingress.annotations
"kubernetes.io/ingress.class") }}
        {{- $_ := set
.Values.ingress.annotations
"kubernetes.io/ingress.class"
.Values.ingress.className}}
        {{- end }}
    {{- end }}
```

Einrückungen - indent, nindent

- **indent:** Fügt eine bestimmte Anzahl von Leerzeichen vor jeder Zeile ein.
- **nindent:** Fügt zusätzlich zum Einrücken auch einen Zeilenumbruch vor dem Inhalt hinzu.
- Syntax:
{{ VALUE | nindent N }}
 - VALUE: Wert der eingerückt werden soll.
 - N: Anzahl der Leerzeichen.

```
metadata:  
  labels:  
    {{- include "mychart.labels" . |  
nindent 4 }}
```

- Das Ergebnis einer Funktion oder eines Werts wird durch das | (Pipe) an die nächste Funktion weitergeleitet.
- Jede Funktion arbeitet auf dem Ergebnis der vorherigen.
- Syntax:
{{ VALUE | FUNCTION }}

```
metadata:  
  labels:  
    app: {{ .Values.app.name | upper  
| quote }}  
..  
{{ .Values.service.type | default  
"ClusterIP" }}
```

Bedingungen in Templates

Bedingte Ausgaben lassen sich mit der if-Anweisung realisieren.

`if`-Anweisungen:

```
{{ if .Values.enabled }}
```

Feature ist aktiviert

```
{{ else }}
```

Feature ist deaktiviert

```
{{ end }}
```

Beispiel aus ingress.yaml.

```
{{- if .Values.ingress.enabled -}}
{{- $fullName := include "test.fullname" .
-}}
{{- $svcPort := .Values.service.port -}}
{{- if and .Values.ingress.className (not
(semverCompare ">=1.18-0"
.Capabilities.KubeVersion.GitVersion)) }}
  {{- if not (hasKey
.Values.ingress.annotations
"kubernetes.io/ingress.class") }}
    {{- $_ := set
.Values.ingress.annotations
"kubernetes.io/ingress.class"
.Values.ingress.className}}
  {{- end }}
{{- end }}
```

Iteration über Listen oder Maps:

```
{{ range .Values.ports }}  
  - containerPort: {{ . }}  
{{ end }}
```

-

Beispiel aus `values.yaml`:

```
ports:  
  - 80  
  - 443
```

.

```
apiVersion: v1  
kind: Service  
metadata:  
  name: {{ include "test.fullname" . }}  
  labels:  
    {{- include "test.labels" . | nindent 4 }}  
spec:  
  type: {{ .Values.service.type }}  
  ports:  
    - port: {{ .Values.service.port }}  
      targetPort: http  
      protocol: TCP  
      name: http
```

Defaults für Parameter definieren

Die `default`-Funktion setzt einen Standardwert:

```
image: {{ .Values.image | default "nginx:latest" }}
```

- Beispiel aus `values.yaml`:

```
image: my-app:1.0
```

.

```
apiVersion: v1
kind: Service
metadata:
  name: {{ default "default-service"
    .Values.service.name }}
spec:
  type: {{ default "ClusterIP"
    .Values.service.type }}
  ports:
    - port: 80
```

Parameter einfordern - required

Sicherstellen, dass ein Wert gesetzt ist:

```
{{ required "Ein Wert wird benötigt!"  
.Values.image }}
```

- Verhindert die Ausführung, wenn der Wert fehlt.

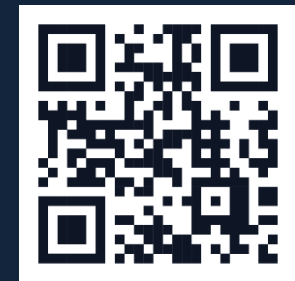
.

```
apiVersion: v1  
kind: Deployment  
metadata:  
  name: {{ required "The deployment name is  
required" .Values.deployment.name }}  
spec:  
  replicas: {{ required "Replica count is  
required" .Values.deployment.replicas }}  
  template:  
    spec:  
      containers:  
        - name: {{ required "Container name is  
required" .Values.container.name }}  
          image: {{ required "Container image is  
required" .Values.container.image }}
```

Praxis

Helm Chart für hashicorp-http-echo

**Vielen Dank für
Ihre Aufmerksamkeit**



ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00
Fax: 0180 1 67349 0

seminare@ordix.de
<https://www.seminare.ordix.de/>